

Scenario 8 — Replay Attack Attempt, Blocked by On-Chain Replay Protection

Full mechanics: see Step-4. This scenario takes the exact same signed message from a normal successful verification and shows, concretely, what happens when someone — not necessarily the original relayer, since submission is permissionless — tries to submit it a second time.

Values used

	Value
Policyholder	Bob, 0x1212...bbbb (Scenario 2's policy)
claimId	2
Oracle signature (legitimate, signed once)	requestId = 0x5e6f7a..., approved = true, timestamp = 1_800_700_000, signature = 0x1c2d3e...
Attacker	0x9999...cccc — note: this address doesn't need any special role at all, since submitVerification is permissionless by design

Walkthrough — the first (legitimate) submission

Identical to Step-4 in every respect: the oracle signs once, someone relays it, `OracleAdapter.submitVerification` verifies the signature, checks `usedRequestIds[0x5e6f7a...] == false` (true, first time), sets it to `true`, forwards to `ClaimRegistry.recordOracleVerification`, which independently checks and sets its own `usedOracleRequestIds[0x5e6f7a...]`. Bob's claim 2 is now `Approved`.

The attack attempt — second submission of the identical signed payload

The Attacker (`0x9999...cccc`) — who was never involved in the original transaction and holds no role on any contract — obtains the same (response, signature) pair (signatures and their payloads are visible on-chain once submitted; nothing about them is secret after the fact) and calls:

`OracleAdapter.submitVerification(response, signature)` — again, with the *exact same* `response = { claimId: 2, requestId: 0x5e6f7a..., approved: true, timestamp: 1_800_700_000 }` and `signature = 0x1c2d3e...` as before.

- **Input:** identical to the first call, byte for byte.
- **Traversal:**
 1. `usedRequestIds[0x5e6f7a...]` — **now true**, set by the first submission.
 2. `if (usedRequestIds[requestId]) revert RequestAlreadyUsed();` — **reverts immediately, on the very first line of logic**. The signature is never even re-verified; the replay check happens before the (more expensive) ECDSA recovery step.
- **Output:** the transaction reverts. Nothing in `ClaimRegistry`'s state changes — Bob's claim status stays exactly as it was after the first, legitimate submission.

Why this can't be exploited to double-pay

Note precisely what replay protection *doesn't* need to worry about here: even if `OracleAdapter`'s own `usedRequestIds` check were somehow bypassed, `ClaimRegistry.recordOracleVerification` performs its own, completely independent check against `usedOracleRequestIds` — this is the deliberate defense-in-depth design covered in Step-4. And even if *both* replay checks were somehow bypassed, `payoutClaim` (Step-5) only pays out a claim whose status is `Approved`, and immediately flips it to `Paid` — a second `payoutClaim(2)` call would separately revert with `ClaimNotApproved`, since the status is no longer `Approved` after the first payout. Three independent layers would all have to fail for a double-payout to actually occur.

End state

<code>claims[2].status:</code>	<code>Approved</code> (unchanged by the attack attempt)
<code>usedRequestIds[0x5e6f7a...]:</code>	<code>true</code> (unchanged - was already true)
Attacker's dUSD balance:	unchanged - the attack never reached any token
transfer at all	